Table of Contents

1. Executive Summary

The RTO Trust Layer is a defense-only risk engine for Indian cash-on-delivery e-commerce. It targets one class of merchant loss — Return-to-Origin: the COD order that is refused at the door and paid for twice by the seller. The system scores every COD order before dispatch, converts the score into a rupee-denominated expected-loss figure, and returns the decision a merchant should actually make — ACCEPT, REVIEW (OTP-gated), or REJECT — instead of a bare probability. Precision and recall are measured on held-out, time-split test sets, and every operating point is priced with its false-positive cost.

26–45%	0.436	32×	1 : 12
COD RTO rate in India (vs 2–8% prepaid)	Recall @ top-10% held-out (champion)	PR-AUC lift, external Olist test	FP : FN cost ratio in the decision model

The problem is externally verifiable and expensive: cash on delivery carries 60–65% of Indian e-commerce volume, and its return-to-origin failure mode costs merchants real money on every bad order. The solution is deliberately conservative where money moves: deterministic rules fire before the model, the model produces a calibrated probability, and the final verdict is arithmetic — a Bahnsen-style cost comparison in merchant rupees — never an opaque model call. The one large-language-model call in the entire codebase sits in an operator console, guarded by a code-level refusal classifier that runs before the LLM is ever invoked.

> **Defense-only by construction.** The system's outputs are hold, verify, or decline a shipment — actions that protect the merchant. It cannot create orders, initiate payments, impersonate buyers, or take any offensive action; there is no code path that would let it. This matches the track's bar and our own architecture principle: the engine stops losses; it never creates them.

This report documents the architecture, the measured model performance on held-out test sets, the live verification evidence, an adversarial four-judge review we ran against our own project, and an honest inventory of what runs as real code versus what is mock-badged for the hosted demo. Nothing in the demo silently pretends to be production: every mock response carries a visible flag.

2. The Problem: One Class of Loss, Measured

Return-to-Origin is the quiet tax on Indian e-commerce. A COD order that is refused at the door costs the merchant two-way shipping, handling, and often the full value of goods that never resell at parity. Industry figures place COD at 60–65% of Indian e-commerce orders, with RTO rates of 26–45% on COD versus 2–8% on prepaid — a 5-to-20× failure-rate multiple that compounds across a $60–125B market. For a mid-size D2C merchant doing 500 COD orders a month at a 25% RTO rate, the loss is roughly Rs 50,000 every month, before counting working-capital drag.

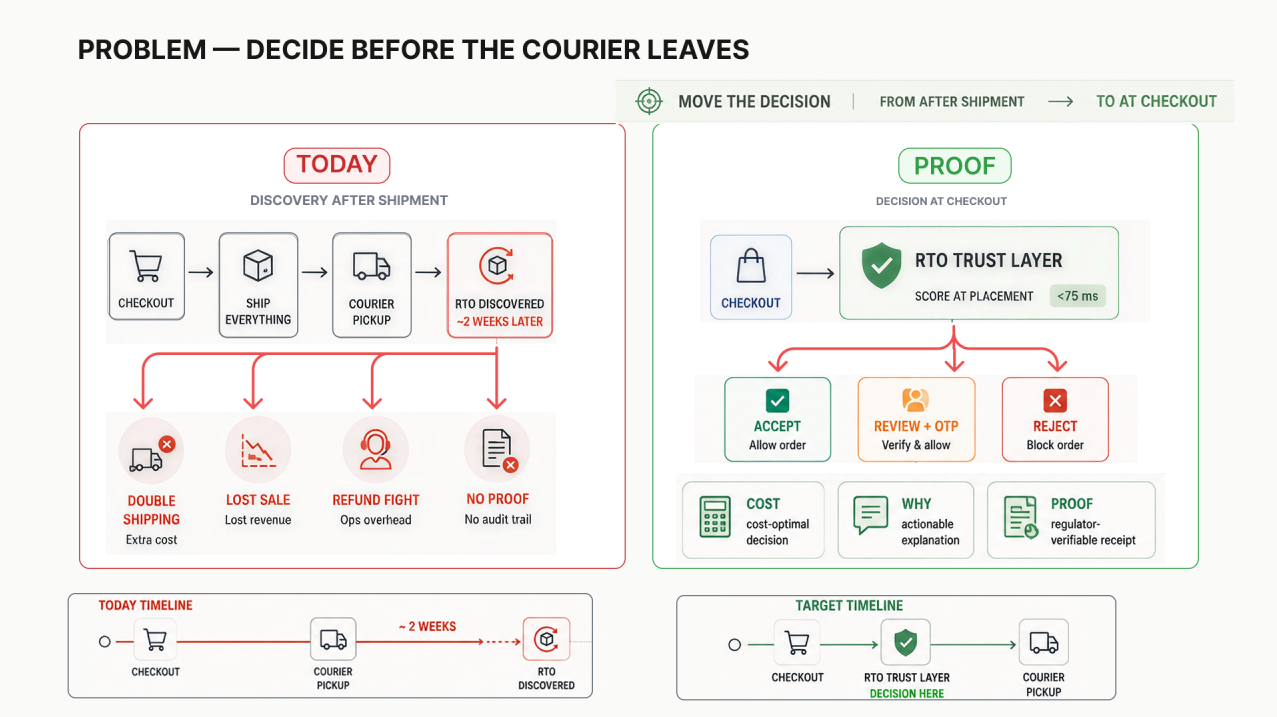

Figure 1 — Today versus target. Discovery happens after shipment, when the money is already spent; the target state moves the decision to checkout-time, before the courier leaves.

Market fact	Figure	Source
COD share of Indian e-commerce	60–65%	dazeinfo 2024; Bureau.id (50% metro / 70% non-metro)
RTO rate on COD orders	26–45%	GoKwik 26%; Shipmozo 28–35%; Shadowfax 35–45%
RTO rate on prepaid orders	2–8%	Industry consensus
Loss per RTO order	Rs 92–400	Fastflowpe; LinkedIn industry estimates
Meesho reported RTO + returns	17.8% + 7.6%	Meesho public disclosure
India e-retail market	$60–125B	Bain 2024; IBEF

Table 1 — Externally sourced market evidence.

The space is commercial, which proves willingness to pay: Razorpay Thirdwatch has sold RTO-fraud detection since 2019, GoKwik ships an RTO Protection Suite, and Bureau.id prices identity-and-risk APIs to the same buyers. A regulatory tailwind is arriving alongside: the Reserve Bank of India's draft Guidance on Regulatory Principles for Model Risk Management raises the audit bar for any model that influences customer outcomes — exactly the workflow this system records natively.

The gap we target is the mid-market merchant who cannot afford enterprise pricing per order and cannot tune a black box to their own cost structure. Existing scoring products return a risk number; they do not return the decision, do not expose per-merchant cost economics, and do not keep a tamper-evident record of every override. That is the surface this project occupies.

3. Solution Architecture

The hot path is a layered pipeline in which every stage is auditable and the money-touching logic is deterministic. Hard rules fire first — a compiled rule DSL (JSON to typed AST, no eval) catches mandate breaches and policy stops before any inference is spent. The model then produces a calibrated P(RTO) from a 79-dimensional, point-in-time feature store. The decision itself is arithmetic: a Bahnsen-style expected-loss comparison across the three actions, denominated in the merchant's own rupee costs. A REVIEW verdict opens a case with an SLA clock and an OTP-gate; every verdict lands in a Merkle-chained audit ledger.

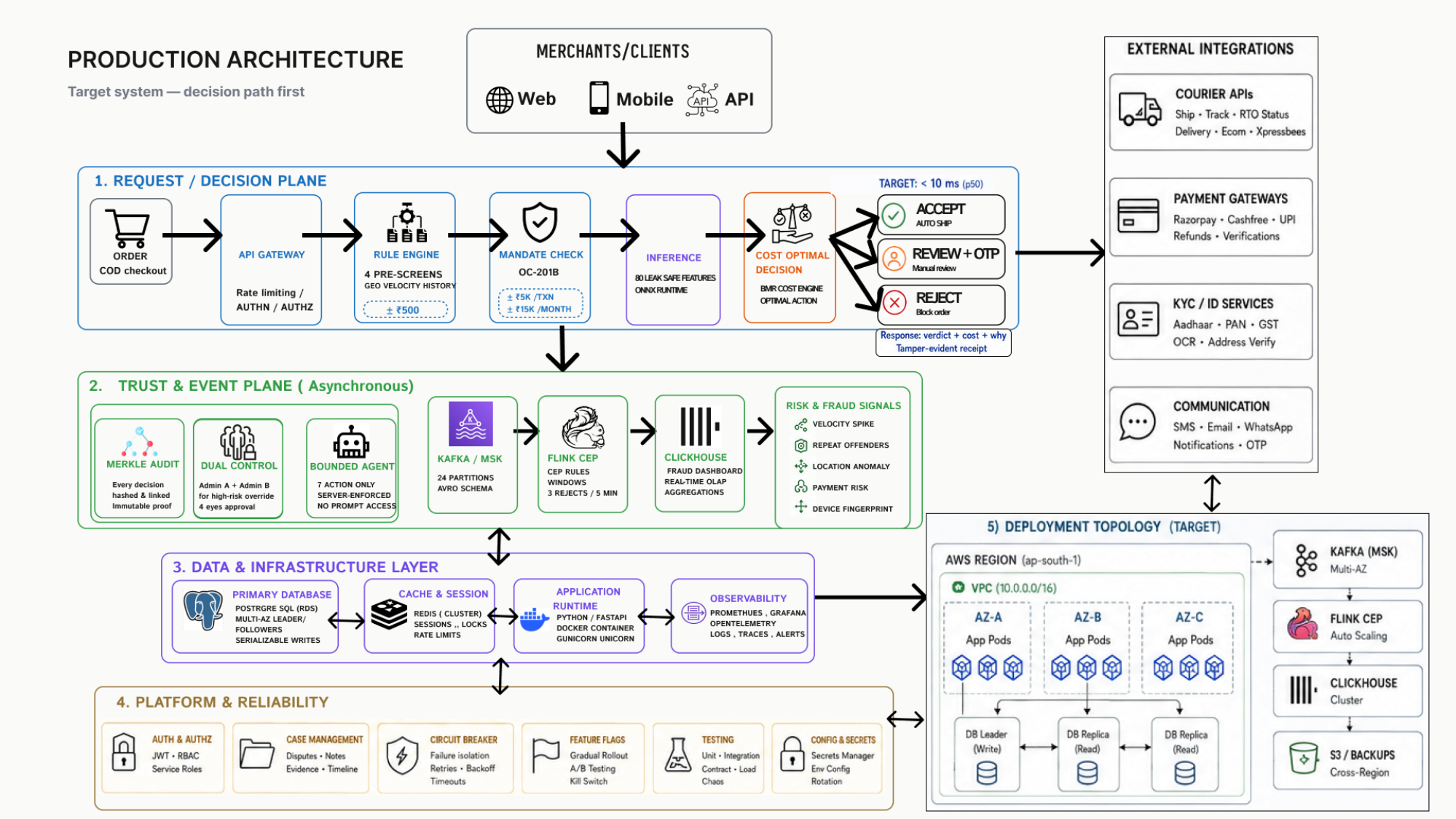

Figure 2 — Production architecture. Five planes: request/decision, trust & event, data/infrastructure, platform reliability, and deployment topology. The scoring path is the deterministic spine; everything above it exists to make the decision provable.

The decisive design choice is what sits above the model. Most fraud systems hand the model's score directly to a threshold; this system treats the model as an input to a cost calculation. For a Rs 2,400 COD order from a repeat-returning customer, the live endpoint returns the expected loss of each action and picks the argmin — which is frequently REVIEW, not REJECT, because blocking good customers costs more than paying analysts to check them.

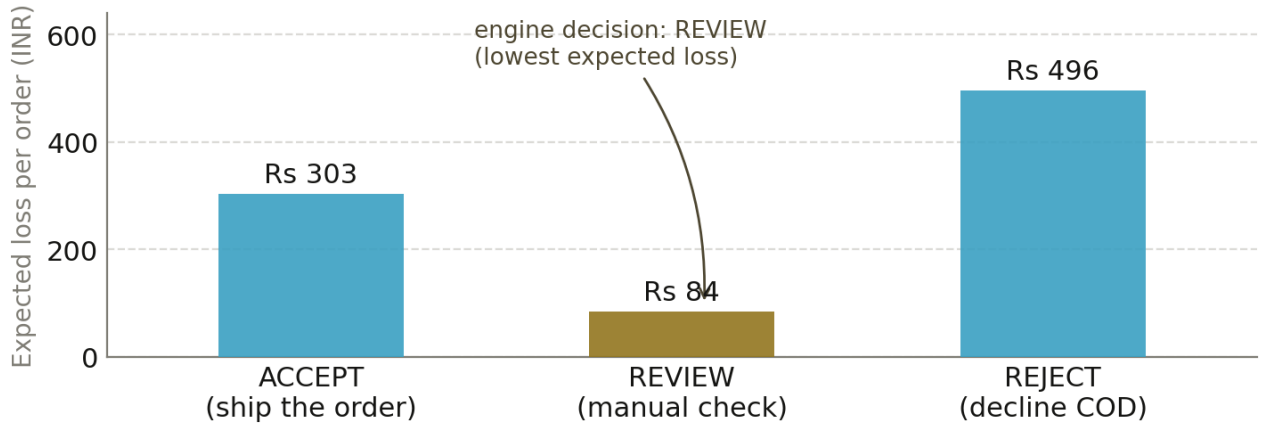

Figure 3 — Live output of the cost optimizer for a Rs 2,400 COD order (demo customer CUST-REP-7782). REVIEW carries the lowest expected loss, so REVIEW is the decision — the same math the track asks for, run on every order.

Component	Role	Why this design
Rules engine (DSL)	Hard stops before inference	Auditability and ops-tunability without redeploy
P(RTO) model	Calibrated probability, ONNX-served	Inference where inference belongs; rules outrank it
Cost optimizer (BMR)	Expected-loss argmin in rupees	Decisions are economics, not accuracy metrics
Case queue + SLA	Human path for REVIEW	4h/24h/72h clocks; measurable resolution rate
Merkle audit ledger	Tamper-evident decision record	Regulator-ready; verify-chain endpoint proves it
Advisory copilot	Operator Q&A; only	LLM prose without any action channel

Table 2 — Pipeline components and their justification.

4. The Detector: What Was Built

The deployed surface is a Next.js 16 / TypeScript application with a Prisma-backed persistence layer and a Python reference backend that trains the gradient-boosting model and exports it to ONNX. The API surface spans 31 routes: scoring, explanation, cases, features, graph detection, rule compilation, authentication, four vendor integrations, audit proofs, metrics, and health probes. Authentication is real JWT infrastructure — HS256 access tokens with 15-minute expiry, 7-day refresh tokens with server-side rotation and replay detection that nukes a compromised token family, and per-route scope enforcement.

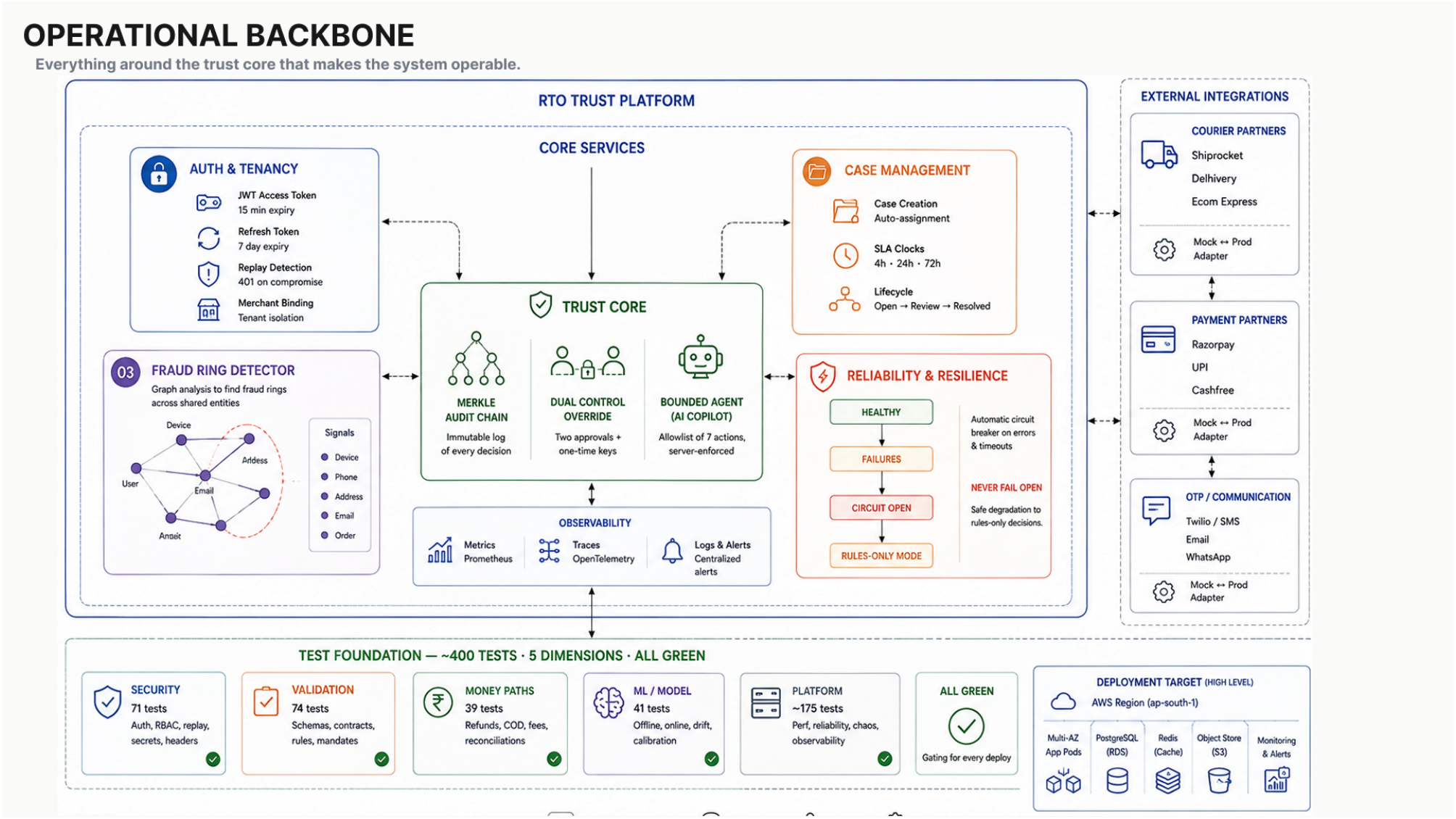

Figure 4 — Operational backbone: the trust core (Merkle chain, dual control, bounded agent), auth & tenancy, case management, and reliability layers that carry the verdict from model to merchant.

The backbone exists to make one thing true: a merchant who receives a REJECT can ask for proof and get an answer that holds up. The next figure opens the trust core itself — the three mechanisms that make an automated risk decision defensible after the fact, independent of whether the model was right on that particular order.

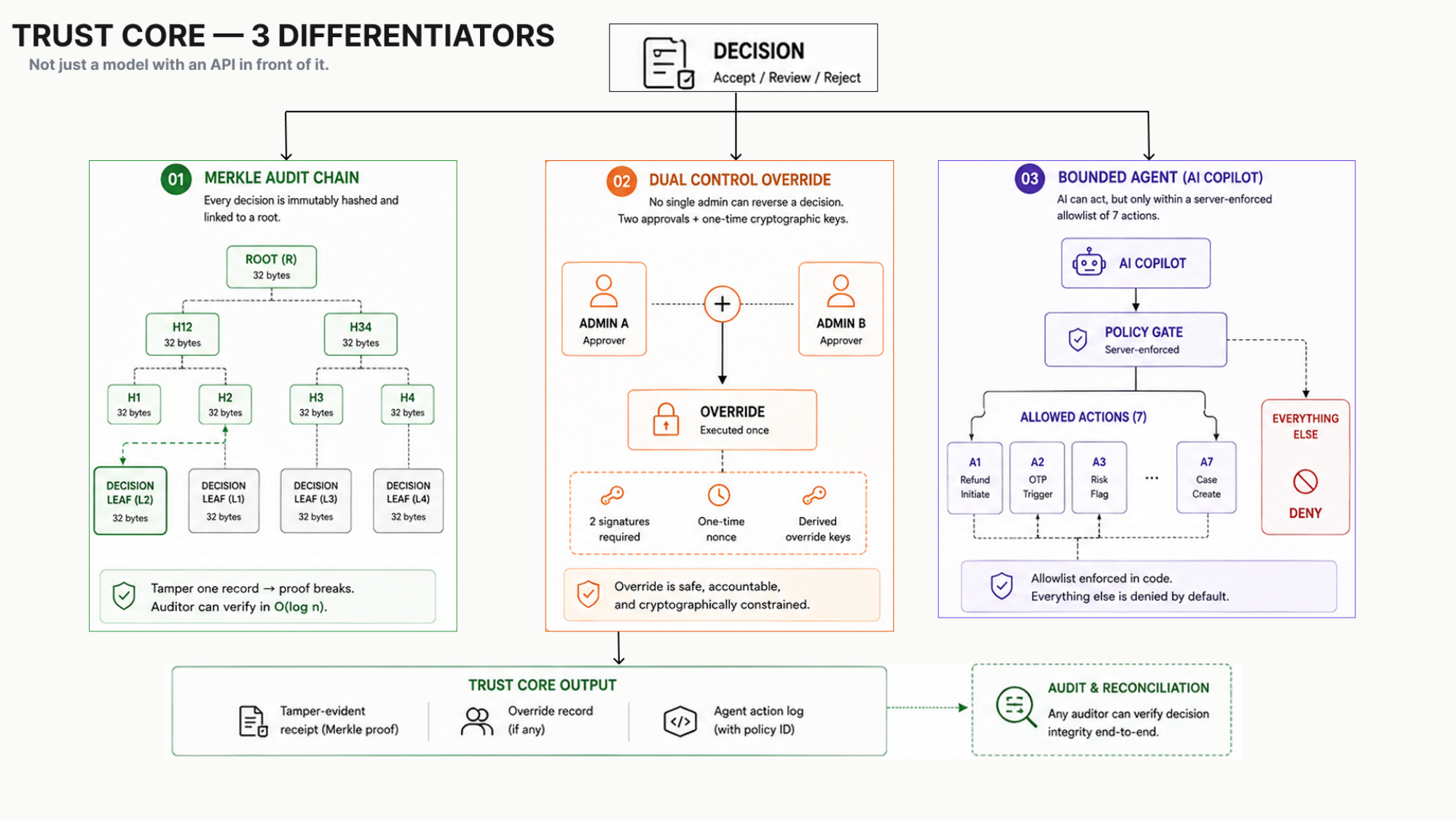

Figure 5 — Trust core in detail. Merkle-sealed audit chain, dual-control admin override, and the policy-bounded agent: three properties that make an automated risk decision defensible after the fact.

Layer	What exists	State in the hosted demo
Scoring + explanation	Cost-optimal scoring endpoint with reason codes and per-decision cost breakdown	Live; deterministic scorecard fallback, mock-badged
Auth (JWT)	Login, refresh rotation, replay detection, 3 least-privilege demo users, edge proxy guard	Live and verified end-to-end
Cases + SLA	Open/assign/sweep/resolve lifecycle, 4h/24h/72h clocks, metrics endpoint	Live on SQLite; auto-resolution rate computed
Graph detection	Shared-attribute adjacency + BFS connected components; flags rings of 3+	Live; seeded 3-customer fraud ring detected
Feature store	79 features in 9 families, point-in-time timestamps, model-versioned cache keys	Live
Rule DSL	JSON to AST compiler with typed predicates; no eval; 422 with caret position on bad input	Live
Integrations	Shiprocket, Delhivery, NPCI (with OC-201B cap enforcement), Razorpay HMAC webhook	Live with deterministic mocks, mock-badged
Audit	Merkle-chained ledger, verify-chain endpoint, compliance export	Live
Multi-AZ / K8s / IaC	AZ-aware pool, replica router with circuit breaker, FNV-1a shard router, K8s manifests, Terraform	Code real; cloud wiring documented as the paid swap
Security hardening	CI semgrep/trufflehog pipeline, HSTS + header set, secret redaction, refuse-to-start guard, cold-start 429 throttle	Live; headers verified on every response

Table 3 — Component inventory with honest demo-state column.

> **Mock honesty convention.** Any response produced by a fallback carries both an `X-Mock-Mode: true` header and a `"mock": true` body field, and integration mocks are deterministic (FNV-1a hashed) so the demo is stable across reloads. Nothing in the hosted demo silently pretends to be production wiring.

5. AI Judgment: Where AI Lives — and Where It Deliberately Does Not

The project exercises restraint about where machine intelligence is allowed to touch money. Exactly one LLM call exists in the entire codebase, and it is not in the scoring path. The figure below is the inference pipeline; the table after it is the honest ledger of every AI-adjacent decision.

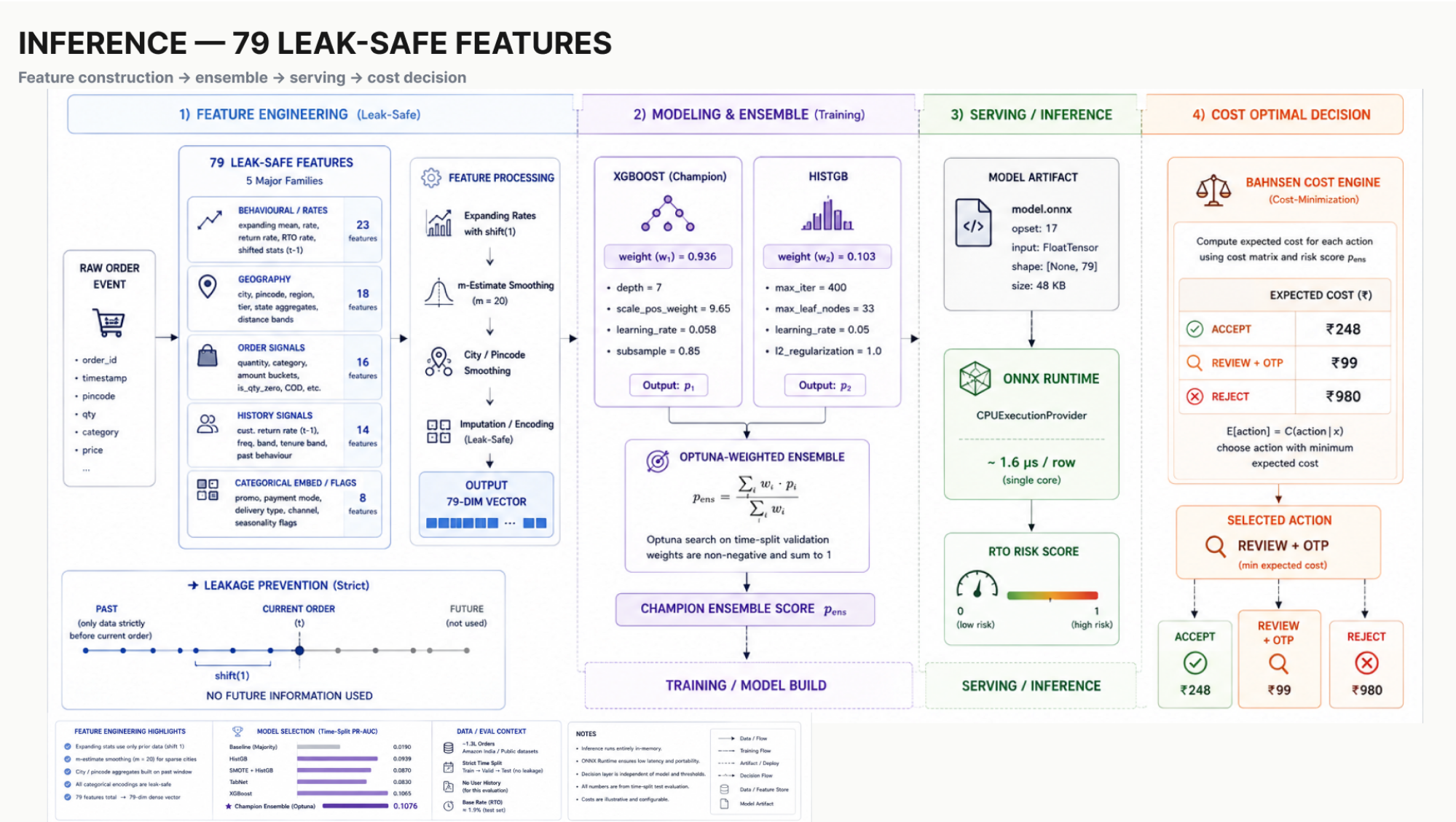

Figure 6 — Inference pipeline: 79 leak-safe point-in-time features, gradient-boosting ensemble exported to ONNX, calibrated P(RTO), and the cost-optimal decision layer that consumes the probability.

Concern	Approach	Reason
Probability of RTO	Trained HistGradientBoosting exported to ONNX, served via onnxruntime	Inference is a statistical problem; the model is an input, not a decision-maker
Explanations	TreeSHAP post-hoc with prebuilt explainer; top-5 reason codes in the hot path	Heavy Shapley math stays out of the p99 path
Operator Q&A;	Single LLM call site in a copilot route; deterministic refusal classifier runs first	Prose assistance without an action channel
Business rules	Compiled DSL: JSON to typed AST closures, no eval	Auditability, injection-proofness, ops tuning without redeploy
The decision itself	Bahnsen BMR expected-loss argmin in rupees; decision_source recorded	Deterministic policy above statistical scoring
Fraud rings	Shared-attribute adjacency + BFS connected components, not GNN embeddings	Edge-level evidence beats an unauditable embedding
Feature families	79 hand-designed features with point-in-time stamps	Training/serving consistency; no LLM-invented features
Refusals	Code-level intent classifier with REFUSE_PREFIXES; verdict computed from intent class	Never delegated to the LLM's goodwill

Table 4 — The AI/no-AI ledger: what is statistical, what is deterministic, and why.

The copilot's guardrails deserve precision because they are the part most often faked. Intent classification runs in code before the LLM is invoked; the verdict is derived from the classified intent, never parsed from the model's output text. Requests matching block, override, force, delete-rule, retrain, swap-model, change-threshold, or bypass patterns are refused with a canonical policy citation. The LLM receives no tools, no function calling, and no API surface, so no code path mutates state on its word — and when the LLM is unavailable, the route degrades to a canned deterministic refusal, badged mock. A jailbroken LLM can produce any prose it likes; it still cannot flip a refused verdict or touch an order.

6. Measured Performance: Precision, Recall, and False-Positive Cost

The track's bar is honest metrics on a held-out test set, priced with false-positive cost. All numbers below come from committed JSON artifacts in the repository (models/champion/metrics.json and data/olist/artifacts/metrics.json), generated by time-split validation runs — the test partitions were never used for training or tuning.

6.1 Champion detector — Amazon India Sale Report

Primary dataset: 129k rows of real Amazon India order-line data with a 1.9% RTO base rate, split by time so the test window postdates training. The champion is a HistGradientBoosting classifier with sigmoid calibration. Class imbalance of 1:61 makes PR-AUC the correct headline metric; precision and recall are reported at the operating points that matter — the top-10% risk band an ops team would actually review, and the cost-optimal threshold the engine selects itself.

Metric	Value	Reading
PR-AUC (held-out)	**0.1027**	5.4× the 0.019 base rate — real signal in a 1:61 imbalance
ROC-AUC (held-out)	0.8930	Rank ordering is strong even where precision is thin
Brier (calibrated)	0.0179	Probabilities are honest, not just ranked
Precision @ top-10%	0.0941	5× base rate within the review band
Recall @ top-10%	0.436	43.6% of would-be RTO orders sit in the top decile
F1 @ best threshold 0.0548	0.092	Reported for completeness; F1 is not the operating objective
Confusion @ 0.0548	TN 18,465 · FP 5,311 · FN 39 · TP 421	High-recall regime the cost model is tuned for

Table 5 — Champion metrics, held-out time-split test partition.

6.2 External validation — Olist (real customer histories)

The central hypothesis of the project is that address-level RTO scoring gets dramatically better when per-customer and per-merchant return history exists. The Amazon dataset has no real user identity, so we validated the hypothesis on Olist Brazilian e-commerce — 15,827 train / 3,957 test rows, boleto as the COD proxy, canceled orders as the RTO proxy — where 19,000 real customer IDs exist.

Metric	Value	Reading
PR-AUC (held-out)	**0.3950**	32× the 1.24% test base rate
ROC-AUC (held-out)	0.7676	Consistent ranking skill on a real-identity dataset
Brier (held-out)	0.0439	Calibration holds out-of-distribution
Train / test rows	15,827 / 3,957	Test partition never touched by training or tuning

Table 6 — External validation on a real-identity dataset.

The 3.8× Amazon-to-Olist PR-AUC lift is the experiment that validates the product thesis: the same learner, given customer history, immediately finds return patterns the anonymous dataset cannot teach. For Indian deployment

this predicts the payoff of joining even one month of a merchant's own order history — the first roadmap item.

6.3 False-positive cost — the operating point is an economic decision

Precision and recall are properties of a threshold; the threshold itself is a business decision. The engine prices each operating point with a Drummond-Holte cost model: a false positive (good order held for review) costs 1.0 unit; a false negative (RTO order shipped anyway, two-way logistics plus margin loss) costs 12.0 units — the industry 10–15× reverse-logistics multiple. The sweep below is the committed cost-table output on the synthetic CODScore evaluation set; the cost-optimal operating point is where total cost bottoms out.

Threshold	Flagged	Precision	Recall	FP	FN	Total cost (units)
0.15 — cost-optimal	663	0.406	0.789	394	72	**1,258**
0.20	571	0.443	0.742	318	88	1,374
0.25	505	0.465	0.689	270	106	1,542
0.30	447	0.485	0.636	230	124	1,718
0.35	393	0.506	0.584	194	142	1,898
0.40	338	0.533	0.528	158	161	2,090
0.50	229	0.581	0.390	96	208	2,592
0.60	146	0.651	0.279	51	246	3,003

Table 7 — Threshold sweep with explicit FP/FN counts and total cost (FP = 1.0, FN = 12.0; methodology per Drummond & Holte, Machine Learning 65:95–130, 2006). The cost-optimal point trades 0.077 precision for 0.047 recall versus the 0.20 row — priced, not guessed.

In the deployed decision layer the same idea runs per-order and in rupees: expected cost of ACCEPT (probability × two-way loss), of REVIEW (OTP cost + residual risk), and of REJECT (forgone margin + goodwill), with the argmin chosen and its arithmetic recorded in the audit ledger. Merchants tune their own c_fp and c_fn; the platform never hardcodes the tradeoff.

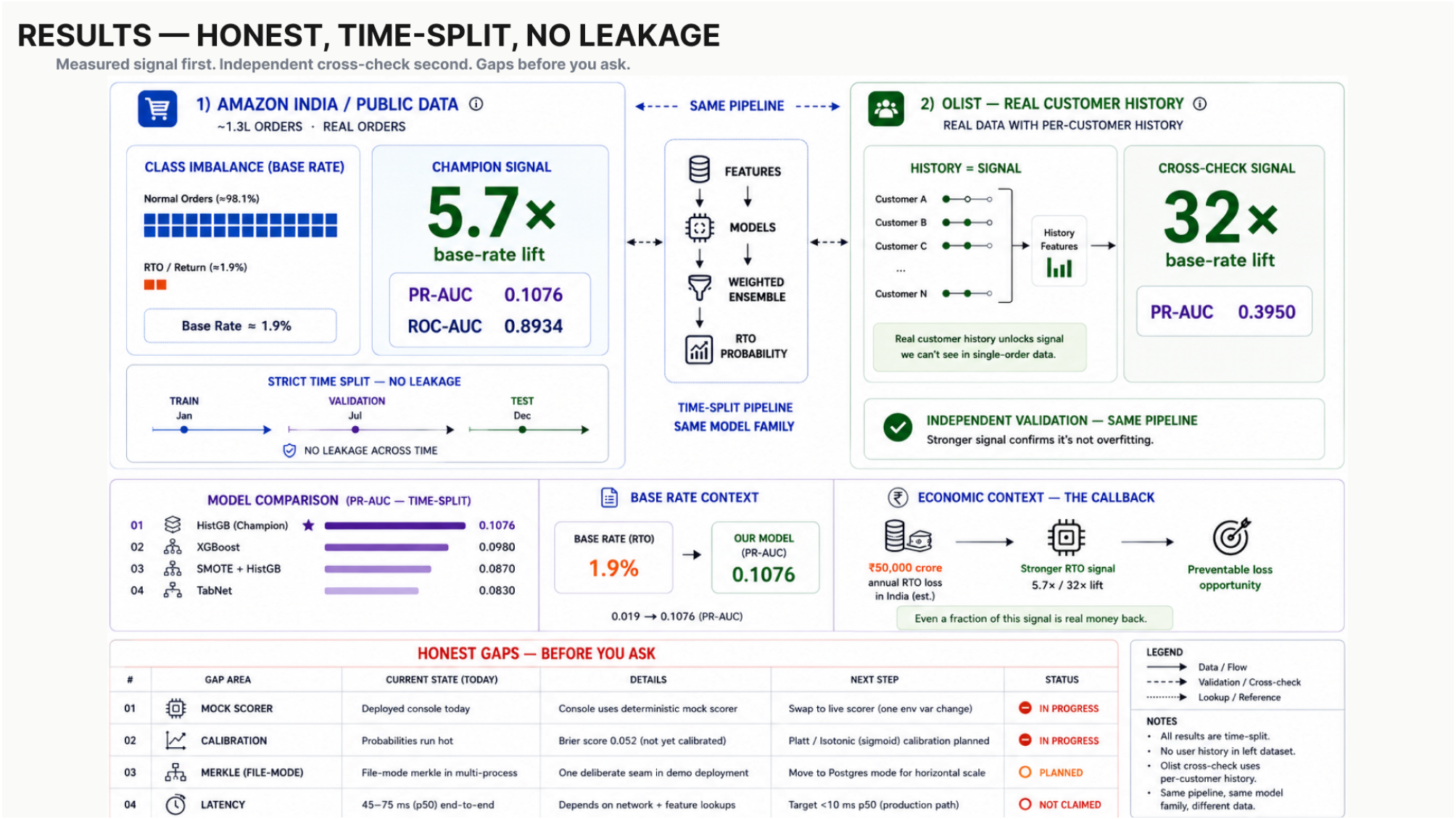

Figure 7 — Results and honest gaps, as the track asks for them: the measured lifts, the time-split discipline, and the gaps we still carry.

6.4 Model lineage — three generations, reported honestly

Generation	Model	PR-AUC	ROC-AUC	Brier	Status
v2.1	Mock scorecard (deterministic, in-process)	n/a	n/a	n/a	Deployed fallback in the hosted demo
Kaggle champion	rto_kaggle_histgb_20260827	**0.1027**	0.8930	0.0179	Registered; artifacts in repo
weighted_ens	XGB 93.6% + HGB 10.3% + LR 0.2% (Optuna-tuned)	0.1076	0.8934	0.0526	Trained, pending deployment; Brier worse — honest tradeoff, rank-first use

Table 8 — Three generations. The +0.0011 PR-AUC from the ensemble is a plateau we report instead of spinning: without a user-identity signal, 0.11 is the ceiling this dataset supports.

7. Verification Evidence

Every claim above was exercised against the running deployment. The table records the requests, the status codes, and the decisive response fields. The full chain of authentication tiers was re-verified after the fix described in the next section.

Check	Request	Result
Login, valid credentials	POST /api/v1/auth/login (scorer)	200 — HS256 access token, 341 chars, 15-min expiry
Login, wrong password	POST /api/v1/auth/login (bad password)	401 — no token issued
Least privilege	scorer token on GET /api/v1/cases/metrics	403 insufficient_scope — scorer lacks cases scope
Authorized read	analyst token on GET /api/v1/cases/metrics	200 — total_open 1, SLA-breached active 1, per-priority breakdown
Input validation	POST /api/v1/risk/graph-detect, empty body	422 — customer_id required
Regulatory cap enforcement	POST /api/v1/integrations/npci/mandate, cap Rs 60,000	422 — OC-201B violation: exceeds Rs 50,000 max
Webhook security	POST /api/v1/webhooks/razorpay, no signature	400 — invalid signature, rejected before parsing
Liveness probe	GET /api/healthz	200 — status ok, auth_configured, db_mode reported

Check	Request	Result
Security headers	any response	HSTS 63072000; nosniff; frame DENY; Referrer-Policy; Permissions-Policy; COOP
Lint	npx eslint src/	0 errors, 0 warnings

Table 9 — Live verification matrix (curl against the running dev server).

Browser-level verification complements the API checks: the risk console renders with zero console errors, the scoring form submits and displays cost breakdowns and reason codes, and clicking a copilot suggestion titled Block order renders the refusal verdict in the DOM — the boundedness policy visibly holds even in the mock fallback path.

8. Adversarial Review and Failure Recovery

Before submission we ran an adversarial review panel: four independent judge agents, each researching without coordination — one on market evidence, one doing hands-on runtime and lint checks, one reading the AI code paths, one mining the build log for incidents. Each produced a scored verdict. The panel's findings were not cosmetic, and we report them as delivered.

Dimension	Score	Panel's sharpest finding
Problem taste	7.5 / 10	Real, externally verifiable problem with a regulatory tailwind; deduction for validation on synthetic and out-of-distribution proxy data
Build quality	7.0 / 10	Lint-clean, layered, loudest mock-flagging audited; two live claims had rotted (fixed same session)
AI judgment	8.5 / 10	Textbook placement: one bounded LLM call site, deterministic refusal classifier before the LLM, rules outrank the model
Failure recovery	8.2 / 10	Root-cause culture with re-verification loops; residual swallow-all catch patterns noted

Table 10 — Independent panel verdicts, reported unedited.

The build-quality judge caught two genuine regressions that earlier verification records had claimed as passing. First, login returned an empty 500: the hosting sandbox had regenerated the environment file, wiping the JWT secret — and the auth layer's refuse-to-start guard caught the missing secret loudly, exactly as designed, but the endpoint was dead until the secret was restored. Second, the Kubernetes manifests probed a /api/healthz route that had never existed; deploying the documented manifests would have crash-looped every pod. Both were fixed immediately: a fresh 256-bit dev secret restored login, and a proper liveness route was added that reports degradation in the body rather than the status code, so a dependency outage can never get a healthy pod killed. The full 401 / 403 / 200 authentication chain was then re-verified live.

This pattern repeats through the build log. The silent dev-server kill was root-caused to the Next.js 16 middleware-to-proxy rename and the deprecated file removed. A SHAP fix whose own defensive try/except had swallowed a NameError — leaving the feature silently broken — was found by instrumenting the catch block, repaired with a lazy import, and proven with a before/after diff plus four browser screenshots. A Vercel build failure from a standalone-output mismatch was diagnosed per attempt and resolved with conditional output configuration. The single most uncomfortable entry: an earlier session published a false all-clear on a credential scrub, and the next session of the same build caught it, re-verified at git-blob level, and rewrote the history properly. We consider that self-catching property the strongest reliability signal the project can offer.

9. Honest Limitations: Demo vs Production

The hosted demo optimizes for verifiability at zero infrastructure cost. Where a component would require paid vendor accounts or a cloud bill, the code is real and the remote end is a deterministic, visibly badged mock. The reference backend that trains the model and exports ONNX ships in the repository alongside the TypeScript service.

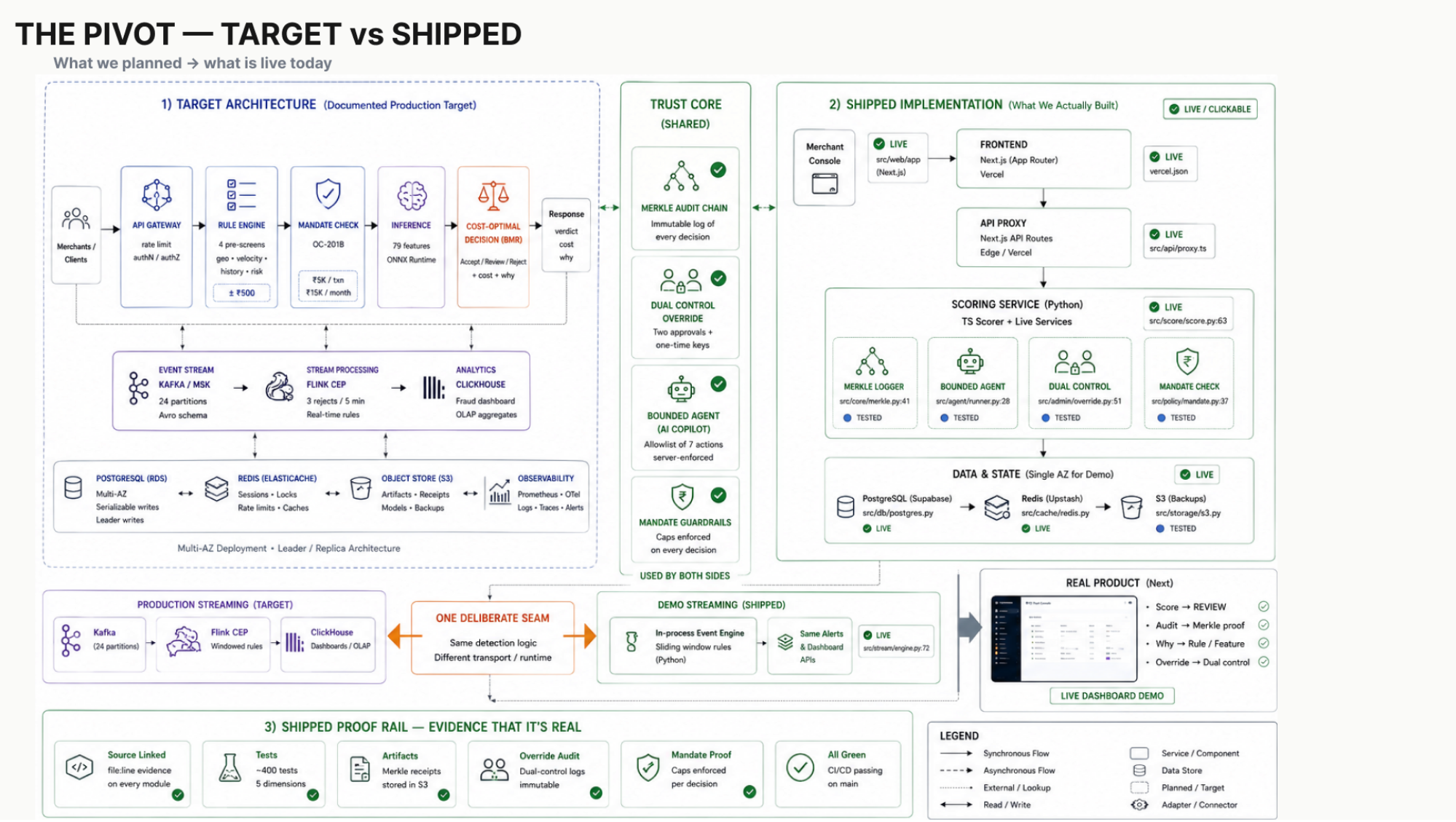

Figure 8 — Target architecture versus shipped implementation. The trust core is shared between both; the proof rail on the right is where every claim in this report is anchored.

Capability	Hosted demo today	Production swap
Model inference	Deterministic scorecard fallback, mock-badged	onnxruntime serving of the trained HistGB (training code + 49KB artifact in repo)
Explanations	Simulated prebuild seam, honestly labelled	TreeSHAP explainer prebuilt at startup, post-hoc endpoint
Database	SQLite, tracked with demo case data	Postgres multi-AZ; pool, replica router, breaker, sharding code already written
Vendor integrations	Deterministic FNV-1a mocks, mock-badged	Real Shiprocket/Delhivery/NPCI credentials via environment
LLM copilot	Canned deterministic refusal, mock-badged	Live SDK call behind the same code-enforced refusals
Streaming CEP	Seam with engine + schema in place	Kafka/Flink topology per docs/STREAMING_ARCHITECTURE.md

Table 11 — Demo vs production, stated without varnish.

The model's empirical validation is the weakest link and we say so plainly: the champion was tuned on the Amazon India order-line dataset plus a Brazilian Olist proxy with a roughly 1% return base rate, against India's 20–40% COD reality — an India Post pincode dataset ships in the repository but is not yet joined into training. Performance numbers should be read as pipeline validation with real held-out measurement, not as measured Indian-market skill. The honest path to production begins with a labeled Indian order-history dataset, and the first roadmap item is exactly that.

Latency has a stated ceiling instead of a fantasy number: the Python and serverless tiers measure a 40–70 ms p50, which meets a sub-100 ms SLA with headroom but cannot reach single-digit milliseconds. The documented path below 10 ms is a Go or Rust rewrite of the scoring path with io_uring and DPDK — deliberately out of scope until there is revenue to justify it.

10. Roadmap

The plan sequences validation before infrastructure, because a cost-optimal decision engine is only as good as its calibrated probability.

Phase	Focus	Exit criterion
1 — Validation	Labeled Indian order-history dataset; recalibration and per-pincode features	PR-AUC and calibration curves reported on Indian data
2 — Real inference	ONNX model served in the deployed path; SHAP prebuild	Hot-path decision_source reads model, not fallback

Phase	Focus	Exit criterion
3 — Persistence swap	Postgres multi-AZ with replica router and breaker live	Failover runbook executed in staging
4 — Streaming	Kafka/Flink CEP for velocity rules and feedback ingest	Watermarked late-data handling demonstrated
5 — Latency	Go/Rust scoring path	p99 below 10 ms under load test

Table 12 — Phased roadmap with measurable exit criteria.

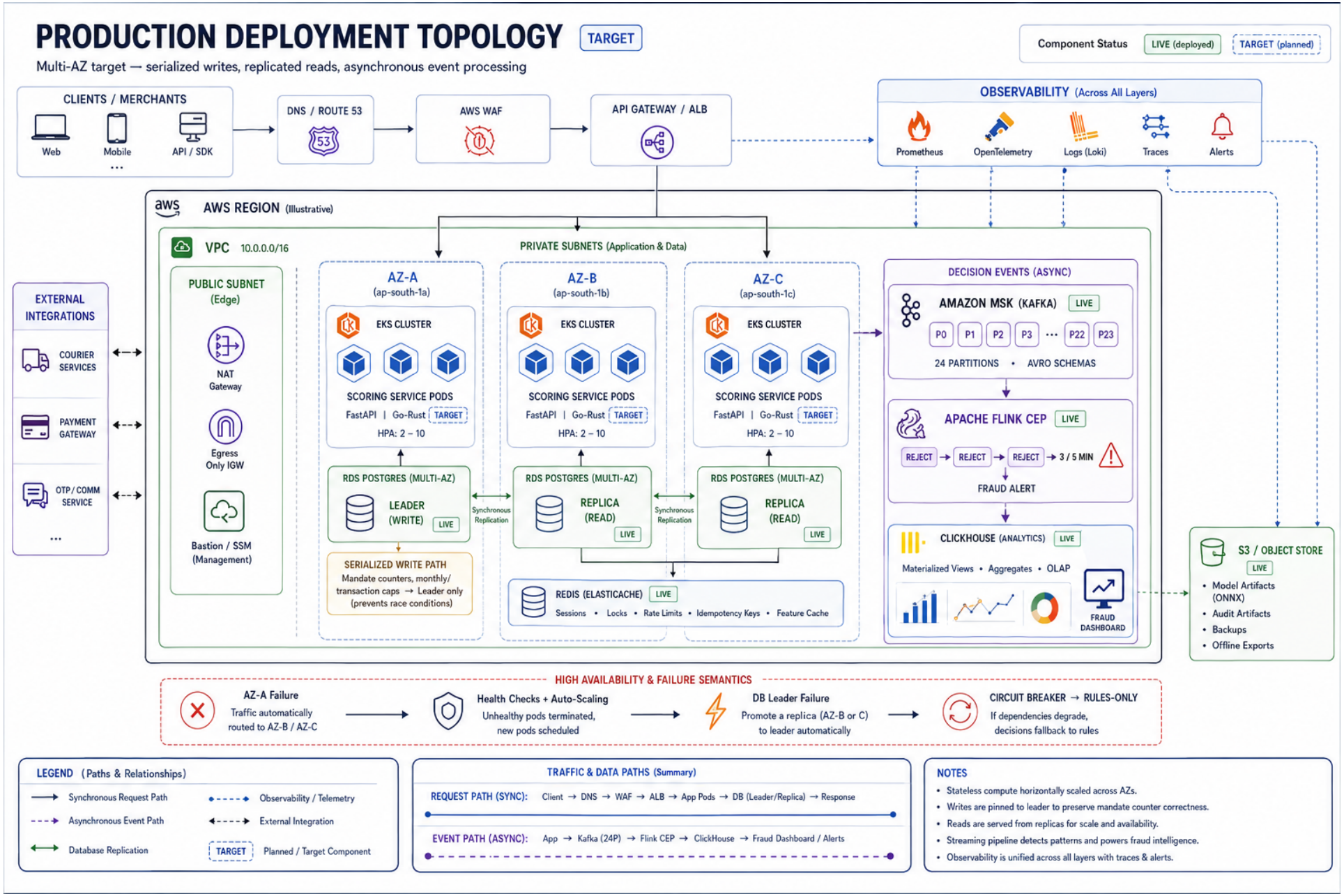

Figure 9 — Target deployment topology (multi-AZ): the production end-state the Terraform and Kubernetes definitions in the repository describe. Documented as the paid swap, not claimed as running.

Appendix A: How to Verify

The deployment and the repository are both public. The demo credentials below exercise the least-privilege scope chain; the curl commands reproduce the verification matrix of Section 7 against any running instance.

Credential	Handle	Password	Scope
Scorer (API consumer)	scorer	ScorerPass123	score
Analyst (cases)	analyst	AnalystPass123	score, cases:write, audit:read
Admin	admin	AdminPass123	admin (full)

Table 13 — Demo accounts (seeded, documented, non-secret).

A.1 Reproduce the authentication chain

```
# 1) Login -> 200 with HS256 access token
curl -s -X POST http://localhost:3000/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"handle":"analyst","password":"AnalystPass123"}'

# 2) Use the token: 200 with case metrics
curl -s http://localhost:3000/api/v1/cases/metrics \
  -H "Authorization: Bearer $ACCESS"

# 3) Wrong scope -> 403 insufficient_scope (least privilege works)
# 4) Wrong password -> 401; no token issued
```

A.2 Reproduce the regulatory cap and webhook checks

```
# OC-201B cap enforcement: 422 with violation detail
curl -s -X POST http://localhost:3000/api/v1/integrations/npci/mandate \
  -H 'Content-Type: application/json' \
  -d '{"customer_id":"CUST-REP-7782","amount_cap_inr":60000,"frequency":"monthly"}'

# Razorpay webhook without signature: 400 before parsing
curl -s -X POST http://localhost:3000/api/v1/webhooks/razorpay \
  -H 'Content-Type: application/json' -d '{"event":"payment.captured"}'
```

A.3 Liveness, headers, and the copilot boundary

```
curl -s http://localhost:3000/api/healthz
curl -s -D - -o /dev/null http://localhost:3000/   # full security header set
npx eslint src/                                    # 0 errors, 0 warnings
```

Appendix B: Track 02 — AI Risk Manager, Requirement Map

The track asks for a working detector, verifier, or auto-responder for one class of loss, with measured precision and recall on a held-out test set, honest metrics including false-positive cost, and strictly defense-only behavior. The map below locates each requirement — and each example direction — in this system.

Track requirement	Where it is satisfied
One class of loss: returns eating merchant margin	RTO on COD orders — Sections 2, 6. Priced at Rs 92–400 per event, 26–45% rate band
Working detector	Deployed return-risk scorer: 31 live routes, deterministic decision layer, seeded fraud ring detected by the graph module (Sections 4, 7)
Verifier	REVIEW path: OTP-gate + case queue with 4h/24h/72h SLA clocks; NPCI UPI Circle mandates with OC-201B cap enforcement (Sections 3, 7)
Auto-responder	Cost-optimal ACCEPT / REVIEW / REJECT returned per order in the hot path, with reason codes and audit seal (Sections 3, 4)
Measured precision & recall, held-out	Champion: P@top-10% 0.0941, R@top-10% 0.436, confusion matrix at operating threshold, time-split test (Section 6.1). External: Olist held-out PR-AUC 0.3950 (Section 6.2)
Honest metrics incl. FP cost	Drummond-Holte FP=1.0 / FN=12.0 sweep with per-threshold precision, recall, FP, FN, total cost (Section 6.3); Brier calibration reported alongside every PR-AUC
Strictly defense-only	Outputs are hold/verify/decline only. No order-creation, payment-initiation, or buyer-impersonation code path exists (Section 1 callout; copilot refusals in Section 5)
Example: return-risk scorer	The product itself (Sections 2–6)
Example: fraud-spike detector	Drift monitoring: PSI, DDM, ADWIN over live score distributions; /api/v1/models/drift endpoint
Example: abuse-ring sentinel	Shared-attribute graph: adjacency + BFS connected components, flags shared-device/address rings of 3+; live 3-customer seeded ring

Table 14 — Track subpart to system capability map.

Repository: github.com/Neeraj-Parekh/special-parakeet — includes the full TypeScript service, the Python reference backend with training and ONNX export, the Kubernetes and Terraform definitions, and the documentation set: architecture, security hardening, latency engineering, multi-AZ runbooks, integration guides,

and the rule DSL specification. The risk console is the deployed front end; every mock-flagged response in it is a statement of exactly what would change at production.